SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I ____K.Niharika(192411124)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

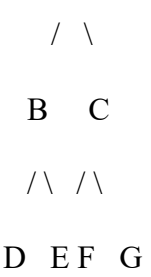
Signature of the student:K.Niharika

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



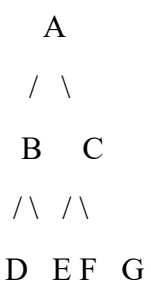
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
   1 2 3
   4 5 6
   7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion
1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Q1. Breadth-First Search (BFS)

Breadth-First Search (BFS) is a graph traversal algorithm used to visit all the nodes of a graph level by level. It starts from a given node and visits all its adjacent nodes before moving to the next level. BFS uses a Queue data structure, which follows the FIFO (First In, First Out) principle.

Given graph: Starting node is A. The adjacent nodes are visited from left to right.

Iteration Table:

Iteration	Queue (after visiting)	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	Empty	G

1. BFS Traversal Order:

A → B → C → D → E → F → G

2. Queue Contents After Each Iteration:

Iteration 1: B, C

Iteration 2: C, D, E

Iteration 3: D, E, F, G

Iteration 4: E, F, G

Iteration 5: F, G

Iteration 6: G

Iteration 7: Empty

Explanation:

- Initially, node A is visited and inserted into the queue.
- Node A is removed, and its adjacent nodes B and C are added to the queue.
- Node B is visited next, and its adjacent nodes D and E are added.
- Node C is then visited, and its adjacent nodes F and G are added.

- Nodes D, E, F and G are visited in order.
- After visiting each node, the next node is removed from the front of the queue.
- The process continues until all nodes are visited and the queue becomes empty.

Result:

The Breadth-First Search algorithm is performed successfully, and the traversal order obtained is A → B → C → D → E → F → G.

Q2. Depth-First Search (DFS)

Depth-First Search (DFS) is a graph traversal algorithm that explores a graph by visiting one branch as deeply as possible before backtracking to explore other branches. It uses a stack data structure, which follows the LIFO (Last In, First Out) principle.

Iteration Table:

Assuming the nodes are explored from left to right, with the leftmost node visited first:

Iteration	Stack (after visiting)	Visited Node
1	B, C	A
2	D, E, C	B
3	E, C	D
4	C	E
5	F, G	C
6	G	F
7	Empty	G

1. DFS Traversal Order:

A → B → D → E → C → F → G

2. Explanation:

- Initially, node A is visited and its adjacent nodes B and C are placed on the stack. Node B is explored first because it is the leftmost child.
- From node B, the algorithm moves to node D, following the left branch as deeply as possible.
- After visiting D, there are no further unvisited children, so the algorithm backtracks to node B and visits E.
- Once both D and E are visited, the algorithm backtracks to A and explores the remaining branch through node C.
- From node C, node F is visited first, followed by node G after backtracking.
- At each iteration, the next node is selected according to the stack's LIFO order. This allows DFS to explore deeper nodes before returning to earlier nodes.
- The process ends when all seven nodes have been visited and the stack becomes empty.

Result:

Thus, the Depth-First Search algorithm is performed successfully on the given graph, starting from node A. The traversal order obtained is A → B → D → E → C → F → G. DFS explores each branch deeply before backtracking, using the stack to keep track of the nodes yet to be explored.

Q3. 8-Puzzle Problem Using Breadth-First Search (BFS)

Initial State:

1	**3**	**6**
5	0	2
4	7	8

Goal State:

1	**2**	**3**
4	5	6
7	8	0

BFS uses a queue that follows FIFO (First In, First Out). At each iteration, the state at the front of the queue is expanded, and its valid, previously undiscovered states are added to the rear.

Iteration table:

To keep the queue readable, each unique puzzle configuration is given a state label, such as S0, S1 and S2. The exact configurations are listed below the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	S0	S0	S1, S2, S3, S4
2	S1	S1, S2, S3, S4	S2, S3, S4, S5, S6
3	S2	S2, S3, S4, S5, S6	S3, S4, S5, S6, S7, S8
4	S3	S3, S4, S5, S6, S7, S8	S4, S5, S6, S7, S8, S9, S10
5	S4	S4, S5, S6, S7, S8, S9, S10	S5, S6, S7, S8, S9, S10, S11, S12

State Configurations:

Each state label represents the following 3 × 3 arrangement.

S0 — Initial state

1	**3**	**6**
5	0	2
4	7	8

S1 — Blank moves Up

1	**0**	**6**

5	3	2
4	7	8

S2 — Blank moves Down

1	**3**	**6**
5	7	2
4	0	8

S3 — Blank moves Left

1	**3**	**6**
0	5	2
4	7	8

S4 — Blank moves Right

1	**3**	**6**
5	2	0
4	7	8

S5

0	**1**	**6**
5	3	2
4	7	8

S6

1	**6**	**0**
5	3	2
4	7	8

S7

1	**3**	**6**
5	7	2
0	4	8

S8

1	**3**	**6**
5	7	2
4	8	0

S9

1	**3**	**6**
4	5	2
0	7	8

S10

1	**3**	**6**
5	3	2
4	7	8

S11

1	**3**	**0**
5	2	6
4	7	8

S12

1	**3**	**6**
5	2	8
4	7	0

1. Which node is expanded in each iteration?

    Iteration 1: S0

    Iteration 2: S1

    Iteration 3: S2

    Iteration 4: S3

    Iteration 5: S4

2. How many states are generated in total?

The complete BFS search discovers 500 unique states, including the initial state, before reaching the goal under the specified move order and duplicate-state handling.

3. Complete solution path:

The shortest solution is obtained in 8 moves. The blank tile moves in the order Right → Up → Left → Down → Left → Down → Right → Right.

Initial State:

1	**3**	**6**
5	0	2
4	7	8

Move 1:

1	**3**	**6**
5	2	0
4	7	8

Move 2:

1	**3**	**0**
5	2	6
4	7	8

Move 3:

1	**0**	**3**
5	2	6
4	7	8

Move 4:

1	**2**	**3**
5	0	6
4	7	8

Move 5:

1	**2**	**3**
0	5	6

4	7	8

Move 6:

1	**2**	**3**
4	5	6
0	7	8

Move 7:

1	**2**	**3**
4	5	6
7	0	8

Move 8 — Goal State

1	**2**	**3**
4	5	6
7	8	0

4. Why does BFS find the shortest solution?

- BFS explores states level by level.
- Every move has an equal cost of one.
- All states requiring fewer moves are explored before deeper states.
- Therefore, the first time BFS reaches the goal, it has found a shortest path.

Result:

BFS systematically explores unique puzzle states using a FIFO queue. The goal state is reached in 8 moves, which is the shortest solution for the given initial configuration.